

Arreglos Sistólicos y Arquitecturas de Dominio Específico

Elly Cortez

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
ecortez@estudiantec.cr*

Juan P. Elizondo

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
juelizondo@estudiantec.cr*

Luis D. García

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
guisogarcia1010@estudiantec.cr*

Yonaikel F. Tencio

*Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Cartago, Costa Rica
tencioyonaikel@estudiantec.cr*

I. INTRODUCCIÓN

Durante décadas, el incremento del rendimiento computacional estuvo impulsado principalmente por procesadores de propósito general cada vez más rápidos. Sin embargo, el crecimiento de aplicaciones como el aprendizaje profundo, el procesamiento masivo de datos y la computación científica ha puesto en evidencia una limitación importante: no todas las cargas de trabajo se benefician por igual de una arquitectura diseñada para ejecutar una amplia variedad de tareas. En muchos casos, la mayor parte del tiempo de ejecución se concentra en un conjunto reducido de operaciones repetitivas, lo que ha motivado el desarrollo de arquitecturas de dominio específico (*Domain-Specific Architectures*, DSA), diseñadas para maximizar la eficiencia sobre un tipo particular de cómputo [1].

Entre las arquitecturas especializadas más influyentes se encuentran los arreglos sistólicos, una organización a nivel de hardware propuesta originalmente para acelerar operaciones matriciales mediante una comunicación local y altamente regular entre elementos de procesamiento [2]. A diferencia de los procesadores tradicionales, donde los datos deben desplazarse constantemente entre memoria y unidades de ejecución, los arreglos sistólicos permiten reutilizar la información conforme atraviesa la arquitectura, reduciendo significativamente el costo asociado al movimiento de datos.

La importancia práctica de este paradigma se ha intensificado en los últimos años, no por una novedad conceptual sino por el tipo de cargas que dominan actualmente los centros de cómputo. Las multiplicaciones de matrices, las convoluciones y los productos punto constituyen la base computacional de la mayoría de las redes neuronales modernas, y todas estas operaciones pueden expresarse en última instancia como secuencias masivas de operaciones *Multiply-Accumulate* (MAC) [1]. Es precisamente este patrón —intenso en aritmética pero también en movimiento de datos— el que los arreglos sistólicos están particularmente bien preparados para resolver, y la razón

por la que numerosas plataformas de aceleración modernas incorporan principios sistólicos en sus diseños.

Antes de analizar implementaciones concretas como los núcleos tensoriales de NVIDIA, las *Tensor Processing Units* (TPU) de Google y las estrategias modernas de optimización de arreglos MAC, resulta necesario comprender los fundamentos arquitectónicos que comparten estas soluciones. En la siguiente sección se introducen el concepto de arreglo sistólico, sus elementos de procesamiento, el flujo de datos que los caracteriza y los mecanismos que les permiten explotar paralelismo masivo en aplicaciones matriciales.

II. FUNDAMENTOS TEÓRICOS

II-A. Origen y concepto

Un arreglo sistólico es una red de elementos de procesamiento idénticos, organizados típicamente en una malla bidimensional, a través de la cual los datos fluyen de manera sincronizada de un elemento al siguiente sin pasar por memoria intermedia [2]. El nombre proviene de una analogía fisiológica: del mismo modo en que el corazón bombea sangre en pulsos regulares, en un arreglo sistólico los datos avanzan ciclo a ciclo, manteniendo un ritmo uniforme de propagación entre vecinos inmediatos.

El concepto fue introducido por H. T. Kung y C. E. Leiserson a finales de la década de 1970, en un contexto orientado al diseño VLSI [1]. La motivación original era práctica: en ese momento el costo dominante de un circuito integrado no era el silicio dedicado al cómputo, sino el cableado necesario para mover datos entre módulos distantes. Restringir las comunicaciones únicamente a vecinos físicos resolvía este problema y permitía construir aceleradores con un *layout* regular, fácilmente replicable y energéticamente eficiente. Décadas después, el resurgimiento del paradigma se debe a que las cargas dominantes del cómputo moderno —multiplicaciones matriciales masivas en redes neuronales— coinciden exactamente con el tipo de operación para el que los arreglos sistólicos fueron concebidos [1].

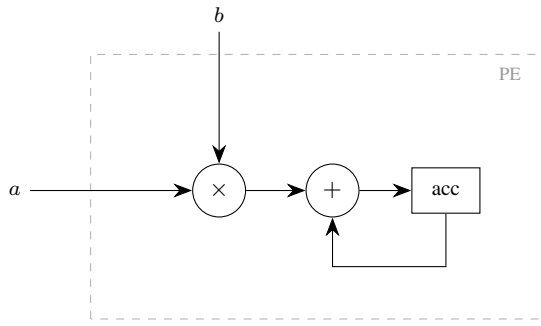


Figura 1: Esquema simplificado de un PE. La operación MAC se implementa con un multiplicador, un sumador y un acumulador con retroalimentación; los caminos de propagación hacia los PEs vecinos no se muestran.

II-B. El elemento de procesamiento

La unidad básica de un arreglo sistólico es el *Processing Element* (PE), un circuito comparativamente sencillo que cumple dos funciones simultáneas durante cada ciclo de reloj. Como nodo de cómputo, ejecuta una operación aritmética básica sobre los operandos que recibe. Como etapa de *pipeline*, propaga esos mismos operandos hacia sus vecinos en la malla. Esta dualidad es esencial: un PE no es solamente un calculador, sino también un eslabón del flujo de datos que recorre el arreglo [2].

Internamente, un PE típico contiene un multiplicador, un sumador, un registro acumulador para almacenar resultados parciales, y registros de entrada y salida que conectan al PE con sus vecinos en las direcciones horizontal y vertical [1]. La Fig. ?? muestra esta organización de forma esquemática. Algunas variantes incluyen además un pequeño espacio de almacenamiento local que permite retener uno de los operandos durante varios ciclos, característica que será relevante al discutir las estrategias de flujo de datos en secciones posteriores.

II-C. La operación MAC

La operación que ejecuta cada PE en cada ciclo se denomina MAC, por *Multiply-Accumulate*. Consiste en multiplicar dos operandos y sumar el resultado a un acumulador interno:

$$\text{acc} \leftarrow \text{acc} + a \cdot b \quad (1)$$

A pesar de su simplicidad, esta operación constituye el núcleo computacional de prácticamente toda carga matricial. Una multiplicación de matrices de dimensión $N \times N$ se descompone exactamente en N^3 operaciones MAC; las convoluciones de las redes neuronales se reducen a sumas ponderadas implementadas también como MACs; y los productos punto en capas completamente conectadas siguen el mismo patrón [1]. Por esta razón los fabricantes de aceleradores miden la capacidad de sus dispositivos en TOPS (*tera-operations per second*), métrica que en la práctica cuantifica el número de MACs ejecutadas por segundo. La eficiencia de un acelerador, en consecuencia, depende no solo de cuántas unidades MAC

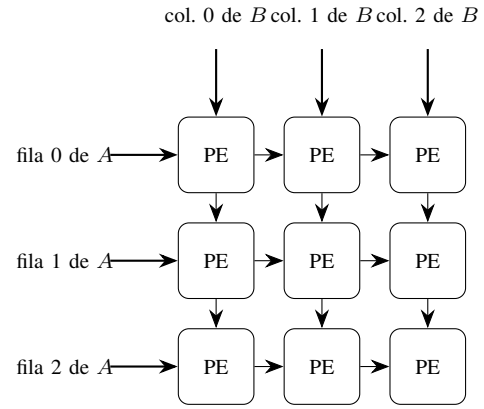


Figura 2: Arreglo sistólico de 3×3 PEs para la multiplicación $C = A \times B$. Las filas de A entran por la izquierda y las columnas de B por arriba.

pueda integrar, sino de qué tan ocupadas pueda mantenerlas a lo largo del tiempo. Las estrategias para lograr esta ocupación sostenida —organización del flujo de datos, ajustes de precisión numérica y balance entre cómputo y memoria— se examinan en detalle en la sección dedicada a la optimización de arreglos MAC.

II-D. Flujo de datos entre elementos

Lo que distingue a un arreglo sistólico de un simple banco paralelo de unidades MAC es precisamente cómo se conectan y comunican sus PEs. En una malla bidimensional para multiplicación matricial, los operandos se inyectan desde dos bordes ortogonales del arreglo y se propagan en direcciones perpendiculares. Para el cómputo $C = A \times B$, los elementos de la matriz A entran por el lado izquierdo y avanzan horizontalmente, fila por fila, mientras que los elementos de B entran por el lado superior y avanzan verticalmente, columna por columna [2]. La Fig. 2 ilustra este patrón sobre una malla de 3×3 PEs.

Esta propagación no es instantánea ni libre. Como los datos tardan ciclos en alcanzar los PEs más alejados de los bordes, las filas y columnas deben introducirse de forma escalonada en el tiempo. En la práctica este escalonamiento se logra insertando ceros al inicio de cada flujo, de modo que el dato correcto llegue al PE correcto en el ciclo correcto [2]. El resultado es lo que en la literatura se denomina un patrón de onda o *wavefront*: una diagonal de actividad que cruza el arreglo desde la esquina superior izquierda hacia la inferior derecha conforme avanzan los ciclos [1].

Cada PE, al recibir simultáneamente un elemento de A y uno de B , ejecuta una operación MAC y luego propaga ambos operandos hacia sus vecinos derecho e inferior. De esta forma, un mismo elemento de A termina siendo utilizado por todos los PEs de su fila antes de salir del arreglo, y un mismo elemento de B por todos los PEs de su columna. Esta reutilización es el verdadero aporte del paradigma sistólico: el dato más costoso —el que viene desde la memoria principal— se amortiza sobre múltiples operaciones aritméticas antes de

descartarse, en lugar de tener que releerse cada vez que se necesita. La forma específica en que las arquitecturas comerciales materializan esta inyección escalonada y la propagación entre PEs —por ejemplo, cómo los núcleos tensoriales de NVIDIA orquestan el avance del frente de onda dentro de un *Streaming Multiprocessor*— se examina en la siguiente sección.

II-E. Paralelismo y multiplicación matricial

El paralelismo de un arreglo sistólico es espacial e inherente a su estructura. Una vez que el frente de onda ha cruzado el arreglo y todos los PEs están alimentados, una malla de $N \times N$ unidades puede sostener hasta N^2 operaciones MAC simultáneas por ciclo [1]. Este crecimiento cuadrático del paralelismo con el tamaño del arreglo es precisamente lo que hace al paradigma tan atractivo para cargas matriciales modernas, donde el número de operaciones también crece de forma cuadrática con la dimensión del problema.

El rendimiento real, sin embargo, nunca alcanza el pico teórico. El frente de onda tarda en llenar el arreglo y, una vez completada la operación, también tarda en vaciarlo, dejando ciclos iniciales y finales con utilización parcial. Este efecto se acentúa cuando las matrices de entrada son demasiado pequeñas para llenar el arreglo o cuando sus dimensiones no coinciden exactamente con la geometría del hardware [1]. Por esa razón, el diseño de un acelerador sistólico no se reduce a elegir un arreglo grande, sino a balancear la cantidad de PEs disponibles, el patrón de alimentación de datos y la geometría típica de las cargas que se desea acelerar.

II-F. Conexión con las arquitecturas modernas

Los conceptos introducidos en esta sección —el PE como unidad básica, la operación MAC como núcleo aritmético, la propagación sincronizada de datos entre vecinos, la reutilización implícita y el paralelismo espacial— constituyen la base común sobre la que se construyen las tres arquitecturas que se examinan en el resto del trabajo. La decisión de qué dato permanece estacionario dentro del PE y cuáles fluyen a través del arreglo define lo que en la literatura se denomina la estrategia de *dataflow*, concepto que reaparecerá de forma explícita en la sección sobre optimización de arreglos MAC.

Los núcleos tensoriales de NVIDIA, descritos en la siguiente sección, incrustan arreglos sistólicos dentro de los multiprocesadores de *streaming* de una GPU, donde conviven con cientos de núcleos CUDA de propósito general. La TPU de Google, por su parte, lleva el principio sistólico al extremo opuesto: dedica casi toda la superficie del *die* a una única gran malla de PEs gobernada por un compilador determinista. Finalmente, las técnicas modernas de optimización de arreglos MAC abordan el problema desde una tercera perspectiva, ajustando la precisión numérica, el flujo de datos y el balance entre cómputo y memoria para extraer mayor eficiencia del mismo principio fundamental. Las tres alternativas son, en esencia, decisiones distintas tomadas sobre el mismo punto de partida que se acaba de describir.

III. NÚCLEOS TENSORIALES

Ante el rápido avance de la inteligencia artificial y del aprendizaje profundo, las unidades de cómputo tradicionales (como los procesadores de flujo convencionales), se volvieron insuficientes para satisfacer la carga computacional requerida por los tensores de gran tamaño (estructuras de datos multidimensionales).

Debido a esta limitante, surgió la necesidad de integrar estructuras de aceleración heterogénea especializadas en operadores específicos. Ante esto, la empresa NVIDIA introdujo por primera vez una unidad de procesamiento de tensores dedicada, conocida como Tensor Core (Núcleo Tensorial). Su propósito era subsanar las deficiencias de cálculo matricial de sus GPUs convencionales, por medio de la aceleración por hardware de precisión mixta, con el objetivo de reducir los tiempos de ejecución al procesar volúmenes masivos de datos.

III-A. El Arreglo Sistólico como el núcleo del Tensor Core

Para comprender cómo los núcleos tensoriales logran esta eficiencia disruptiva frente a las unidades tradicionales de la GPU, es crucial vincularlos con el concepto de los arreglos sistólicos (systolic arrays). Un sistema de cómputo tradicional se ve drásticamente limitado por los constantes accesos y transferencias hacia la memoria principal. En contraste, un núcleo tensorial moderno no reinventa las matemáticas del álgebra lineal, sino que adopta la estructura paralela de un arreglo sistólico bidimensional como su propio núcleo de procesamiento computacional.

La arquitectura del núcleo tensorial aprovecha precisamente la ventaja competitiva de los arreglos sistólicos: una malla o red bidimensional de Unidades de Procesamiento (PE, Processing Units) interconectadas directamente a nivel de hardware mediante una segmentación (pipeline) balanceada.

III-B. La operación MMA

Los Tensor Cores están diseñados específicamente para resolver la ecuación de multiplicación y acumulación de matrices (MMA, Matrix Multiply-Accumulate) [3], según muestra la ecuación 2:

$$D = A \times B + C \quad (2)$$

En este proceso, A y B son las matrices que se multiplican entre sí. Las cuales suelen estar en formatos de baja precisión como FP16 (16 bits de punto flotante) o INT8 (8 bits enteros), lo que permite una mayor velocidad de procesamiento.

La matriz C es la matriz de acumulación, generalmente manejada con una precisión mayor como FP32 (32 bits de punto flotante) para evitar la pérdida de información y mitigar los errores de redondeo.

Por último, el resultado final de la operación se entrega en la matriz D , también de alta precisión.

III-C. Flujo de ejecución en el Hardware

Las matrices de los modelos de aprendizaje profundo, son muy grandes, por lo que un núcleo tensorial no puede procesarlas de golpe. En su lugar, se puede aplicar una estrategia jerárquica de división y ejecución, basada en el flujo de datos de un arreglo sistólico [3].

III-C1. Segmentación por bloques: Las matrices de gran tamaño (A , B y C) se fragmentan en sub tareas de dimensiones fijas que coinciden exactamente con el tamaño físico del arreglo de hardware.

III-C2. Distribución escalonada de datos: La manera en la que los datos son introducidos al núcleo tensorial, no es en forma de un bloque completo, sino que se introducen de manera síncrona y escalonada, por filas y columnas. Lo anterior implica que cada elemento de la matriz entra al circuito con un retraso de un ciclo de reloj respecto a su fila o columna vecina, lo que crea una onda de datos coordinados.

III-C3. El flujo sistólico y el reúso de datos: Aquí es donde entra en juego la magia de los arreglos sistólicos. Una vez que los datos se encuentran dentro de la cuadrícula bidimensional (de PEs), estos fluyen siguiendo el ritmo de un "pulso cardíaco". De esta forma, cada unidad PE recibe un elemento de la matriz A por la izquierda y un elemento de la matriz B desde arriba, realizando la multiplicación de los dos elementos individuales y sumándoles un valor acumulado intermedio; inmediatamente después, la PE pasa el elemento de A y B de la misma forma que como se mencionó al inicio.

III-C4. Acumulación y resultado final: Cada unidad PE aprovecha su registro de acumulación para ir almacenando el resultado de las multiplicaciones parciales. En la práctica, se suele "precargar" la matriz C antes de la entrada de los datos provenientes de las matrices A y B , de esta manera tras sumar cada valor de la matriz C y los valores acumulados de las multiplicaciones parciales, el resultado final de la matriz D se encuentra consolidado.

III-D. El problema de la subutilización

Un problema en la aplicación de los núcleos tensoriales, aparece al momento de considerar todo el sistema arquitectónico que los rodea, ya que esto suma limitaciones al sistema que genera una severa subutilización de los núcleos tensoriales [4]. Lo anterior se puede explicar por tres razones principalmente.

- **La asimetría de la jerarquía de memoria:** Las operaciones de multiplicación de matrices generales (GEMM), requieren un flujo constante y masivo de datos. Mientras que los *tensor cores* pueden procesar los datos casi instantáneamente a nivel de registros, el tiempo que toma mover los datos desde la memoria global hasta la memoria compartida, y luego a los registros de los hilos, es órdenes de magnitud más lento.
- **Dependencias de datos e instrucciones (Latencia del Pipeline):** Las instrucciones de "soporte" provenientes del software de la GPU, llegan a saturar las unidades de ejecución, lo que provoca que la emisión de instrucciones hacia el *tensor core* sea intermitente.

- **Acoplamiento rígido de recursos:** Como los núcleos tensoriales están asignados fijamente a un número específico de unidades de procesamiento, en caso de que el grupo de hilos se llegue a retrasar, ya sea buscando datos en memoria o resolviendo una bifurcación de código, el *tensor core* queda congelado, desperdiciando ciclos de reloj valiosos.

III-E. La dimensión de ejecución y el rol de los warps

El *Warp* es el bloque básico de ejecución en el modelo SIMT de las GPU, el cual consiste en un grupo de hilos que ejecutan la misma instrucción de forma simultánea, sobre diferentes datos [4].

Bajo este nivel de abstracción, el programador de GPU escribe el código pensando en hilos individuales; no obstante, un solo hilo de la GPU no puede controlar un núcleo tensorial. Además, la operación matemática MMA se expone al sistema como una instrucción intrínseca a nivel de Warp. Al final, ningún hilo posee la matriz completa; cada hilo guarda en sus registros privados una pequeña porción de las matrices de entrada A y B .

Una solución a la subutilización de los núcleos tensoriales, es que múltiples Warps compartan el mismo *tensor core* de manera dinámica, esto mediante un planificador especializado a nivel de arquitectura. De esta forma, mientras un Warp se encuentra en espera, el hardware conmuta de inmediato el contexto para que otro Warp totalmente independiente tome el control del núcleo tensorial e inicie su propio cálculo [4].

IV. TPU DE GOOGLE

Ante el crecimiento exponencial de las cargas de trabajo de inferencia en centros de datos, Google diseñó su propia Unidad de Procesamiento Tensorial (TPU, *Tensor Processing Unit*): un circuito integrado de aplicación específica (ASIC) orientado a acelerar operaciones de álgebra lineal densa propias del aprendizaje profundo [5].

A diferencia del enfoque adoptado por NVIDIA, donde el arreglo sistólico coexiste con cientos de núcleos CUDA de propósito general, la TPU subordina toda la superficie del die al cómputo matricial, eliminando los mecanismos de ejecución general que, como se describió en la sección anterior, son fuente de subutilización en los *Tensor Cores*.

IV-A. Motivación y origen

El diseño de la TPU respondió a una proyección concreta: en 2013, Google determinó que si el 10 % de sus usuarios realizara búsquedas por voz durante tres minutos al día, empleando redes neuronales profundas, la demanda computacional de sus centros de datos se duplicaría, lo que implicaría una inversión en infraestructura inviable con CPU convencionales [5]. Esto motivó el desarrollo de un ASIC de alta eficiencia energética bajo la premisa de mejorar el rendimiento por vatio en al menos un orden de magnitud frente a las GPU disponibles en ese momento.

La premisa central de diseño que diferencia a la TPU de una GPU es la siguiente: dado que más del 90 % del tiempo

de inferencia en redes neuronales se destina a multiplicaciones matriciales [6], el hardware debe maximizar la eficiencia de esa operación en particular, en lugar de mantener capacidades de ejecución general costosas en área y potencia.

IV-B. Arquitectura interna: el núcleo sistólico

El corazón de la TPU es la *Matrix Multiply Unit* (MXU), un arreglo sistólico bidimensional de unidades MAC (*Multiply-Accumulate*) organizadas en una malla de procesadores (PE) interconectados localmente [5].

A diferencia de los *Tensor Cores* de NVIDIA —donde el arreglo sistólico es uno de los muchos componentes de un SM (*Streaming Multiprocessor*)— en la TPU el MXU ocupa cerca de un cuarto del área total del die, y la mayor parte del área restante corresponde también a estructuras de soporte de datos.

El flujo de datos dentro del MXU sigue la estrategia *Weight Stationary* (WS): los pesos de la red neuronal se precagan en los registros de cada PE y permanecen fijos durante toda la operación matricial, mientras que los vectores de activación fluyen diagonalmente a través del arreglo, generando una onda de cómputo sincronizada ciclo a ciclo [5].

Este mecanismo es precisamente el flujo sistólico descrito en la sección anterior, aplicado ahora en un ASIC donde no existe ningún overhead de despacho de instrucciones ni gestión de *warps* que interrumpa el cómputo matricial.

No obstante, trabajos recientes han identificado que el dataflow *Weight Stationary* introduce penalizaciones de rendimiento y energía asociadas a los buffers FIFO que sincronizan la entrada y salida del arreglo [7].

Esta limitación ha motivado investigaciones sobre dataflows alternativos orientados a eliminar dichos buffers, como el propuesto por Abdelmaksoud et al. [7], que reporta mejoras de hasta $1.81\times$ en eficiencia energética y $1.49\times$ en latencia frente a arquitecturas tipo TPU en cargas de trabajo de transformers.

Complementando al MXU, la TPU incorpora las siguientes unidades de memoria on-chip:

- **Unified Buffer (UB):** Memoria SRAM de gran capacidad gestionada explícitamente por el compilador, que almacena las activaciones entre capas consecutivas. A diferencia de una caché convencional, cuyo comportamiento es determinista, el UB es controlado directamente por el compilador XLA, lo que elimina el tráfico impredecible a DRAM que limita el rendimiento de las GPU en cargas de inferencia [5].
- **Acumuladores:** Registros de alta precisión (32 bits) que reciben los productos parciales generados por el MXU en cada ciclo de reloj, acumulando el resultado de la operación $D = A \times B + C$ sin necesidad de escrituras intermedias a memoria.
- **Weight FIFO:** Cola de doble buffer que precarga los pesos desde la memoria DRAM externa hacia el MXU, ocultando la latencia de acceso a memoria y manteniendo el arreglo sistólico ocupado de manera continua [5].

IV-C. Modelo de programación: ISA de dominio específico

Una diferencia arquitectónica fundamental entre la TPU y una GPU radica en su modelo de programación. Mientras que una GPU expone una ISA de tipo SIMT con cientos de instrucciones de propósito general gestionadas mediante *warps*, la TPU v1 adoptó una ISA de tipo CISC con aproximadamente una docena de instrucciones de alto nivel orientadas exclusivamente al cómputo de redes neuronales [5].

Las instrucciones fundamentales son cinco:

1. **Read_Host_Memory:** transfiere datos de activación desde la memoria del servidor anfitrión hacia el Unified Buffer.
2. **Read_Weights:** carga los pesos desde la DRAM externa hacia el Weight FIFO.
3. **MatrixMultiply/Convolve:** ordena al MXU ejecutar una multiplicación matricial o una convolución, tomando como entradas el UB y el Weight FIFO, y depositando los resultados parciales en los Acumuladores.
4. **Activate:** aplica la función de activación no lineal (ReLU, Sigmoid, etc.) sobre los Acumuladores y escribe el resultado en el Unified Buffer.
5. **Write_Host_Memory:** transfiere el resultado final desde el Unified Buffer de vuelta a la memoria del servidor anfitrión.

Este modelo determinista evita exactamente el problema de subutilización descrito en la sección anterior: al no existir *warps*, bifurcaciones de código ni cachés automáticas, el MXU opera sin interrupciones durante toda la duración de la instrucción *MatrixMultiply*, cuya latencia típica es de cientos a miles de ciclos de reloj.

IV-D. Evolución hacia la TPU v4: supercomputador reconfigurable

La cuarta generación de la TPU, presentada en ISCA 2023, representa un salto cualitativo en la escala y las capacidades del sistema [5]. Cada chip TPU v4 contiene dos *TensorCores* (TC); cada TC agrupa cuatro MXU de 128×128 MACs, una unidad de procesamiento vectorial (VPU) de 128 carriles, y una memoria vectorial (VMEM) de 16 MiB. Los dos *TensorCores* comparten una *Common Memory* (CMEM) de 128 MiB, lo que amplía considerablemente la capacidad de datos on-chip frente a generaciones anteriores.

La innovación más significativa de la TPU v4 a nivel de sistema es la incorporación de *Optical Circuit Switches* (OCS): conmutadores ópticos basados en espejos MEMS (*Micro-Electro-Mechanical Systems*) que reconfiguran dinámicamente la topología de interconexión entre los chips en milisegundos [5]. Esto permite escalar el supercomputador a 4096 chips organizados en un toro 3D, cuya geometría puede adaptarse por tarea de entrenamiento para maximizar el ancho de banda de comunicación colectiva según el tipo de paralelismo empleado (de datos, de modelo o de pipeline).

Los OCS representan menos del 5 % del costo total del sistema y menos del 3 % de su consumo de potencia, pero aportan cuatro beneficios operativos fundamentales: disponibilidad (se redirige el tráfico alrededor de nodos fallidos),

modularidad (cada rack de 64 chips puede ponerse en producción de forma independiente), flexibilidad topológica (soporte para toro torcido (*twisted torus*), que mejora el rendimiento todo-a-todo en $1.31\times$ – $1.63\times$ frente al toro rectangular), y aislamiento de red entre usuarios distintos que comparten el supercomputador [5].

En cuanto a rendimiento, la TPU v4 supera a la TPU v3 en $2.1\times$ y mejora la eficiencia energética en $2.7\times$. Comparada con sistemas de tamaño similar, es $1.2\times$ – $1.7\times$ más rápida y consume $1.3\times$ – $1.9\times$ menos potencia que la NVIDIA A100 [5].

IV-E. *SparseCores: soporte hardware para embeddings*

Con la proliferación de los modelos de recomendación (*Deep Learning Recommendation Models*, DLRM) y los transformers, el perfil de carga de los centros de datos de Google evolucionó significativamente: mientras que en 2016 el 61 % de los ciclos TPU se destinaban a perceptrones multicapa (MLP), en octubre de 2022 el 57 % correspondía a transformers [5].

Los modelos DLRM dependen de operaciones de *embedding*: búsquedas dispersas (*sparse lookups*) en tablas de gran tamaño que pueden alcanzar varios TiB en agregado. Estas operaciones tienen baja intensidad aritmética y patrones de acceso a memoria irregulares, lo que las hace inadecuadas para el MXU, cuya eficiencia depende de la densidad de las operaciones matriciales [5]. Colocarlas en las CPU anfitrionas introduciría un cuello de botella de ley de Amdahl agravado por la proporción de 4:1 entre chips TPU y hosts CPU.

La solución adoptada por Google a partir de la TPU v2 es el *SparseCore* (SC): un procesador de flujo de datos especializado en operaciones de gather/scatter dispersas, integrado en el mismo die que el TensorCore pero con circuitería optimizada para accesos irregulares a HBM [5]. Usando solo el 5 % del área y la potencia del die, el SparseCore acelera los modelos con embeddings entre $5\times$ y $7\times$ respecto a ejecutarlos en el TensorCore, resolviendo el desbalance de carga que de otra forma limitaría la escalabilidad de los DLRM en el supercomputador.

Este enfoque de integrar unidades heterogéneas especializadas en un mismo ASIC —un TensorCore denso para transformers y un SparseCore para embeddings— ilustra la tendencia general en el diseño de arquitecturas de dominio específico: en lugar de optimizar una única unidad de cómputo, se diseña un conjunto de bloques cada uno especializado para la clase de operación que domina su segmento de la carga de trabajo [5].

Investigaciones recientes han explorado extensiones de este principio al nivel del arreglo sistólico, buscando co-diseñar la estructura del hardware con las propiedades de los modelos, como el co-diseño entre dimensiones del arreglo y bloques de *pruning* estructurado propuesto por Palacios et al. [8], que reporta aceleraciones de hasta el 44 % en sistemas edge con solo un 1.4 % de degradación en la tasa de error.

V. OPTIMIZACIÓN DE ARREGLOS MAC

V-A. *Rol de las operaciones MAC en cargas matriciales*

Las operaciones de multiplicación-acumulación, conocidas como MAC por sus siglas en inglés (*Multiply-Accumulate*), son una de las operaciones fundamentales en aceleradores para inteligencia artificial. Una operación MAC realiza una multiplicación entre dos datos y acumula el resultado sobre una suma parcial. De forma general, puede representarse como:

$$y = y + a \cdot b \quad (3)$$

Esta operación aparece de manera repetitiva en cargas matriciales, como la multiplicación de matrices, las capas completamente conectadas y las convoluciones utilizadas en redes neuronales profundas. Por ejemplo, en una multiplicación matricial, cada elemento de la matriz de salida se obtiene acumulando productos entre una fila de la primera matriz y una columna de la segunda matriz. Por esta razón, una gran parte del tiempo de cómputo en este tipo de aplicaciones se concentra en ejecutar muchas operaciones MAC de forma eficiente [9].

En los arreglos sistólicos, estas operaciones se distribuyen entre varios elementos de procesamiento o PEs. Cada PE puede ejecutar operaciones aritméticas simples, especialmente multiplicaciones y acumulaciones, mientras los datos avanzan de forma ordenada a través del arreglo. Así, en lugar de resolver una carga matricial de manera secuencial en un procesador general, el arreglo permite que muchas operaciones MAC se ejecuten en paralelo [9].

Desde el punto de vista arquitectónico, optimizar arreglos MAC no significa únicamente aumentar la cantidad de unidades de cómputo. También implica organizar adecuadamente cómo se distribuyen las operaciones, cómo se alimentan los PEs con datos y cómo se acumulan los resultados parciales. Esta idea también aparece en implementaciones recientes, donde la multiplicación y la suma se identifican como los principales costos computacionales durante la inferencia de redes neuronales, por lo que el diseño eficiente de unidades MAC resulta clave para mejorar el rendimiento del acelerador [10].

V-B. *Paralelismo y reutilización de datos*

Una de las razones por las que los arreglos MAC son eficientes es que permiten explotar paralelismo espacial. En lugar de ejecutar una operación MAC tras otra de forma secuencial, el arreglo distribuye el trabajo entre múltiples elementos de procesamiento, de manera que varios productos y acumulaciones pueden calcularse al mismo tiempo. Esto es especialmente útil en multiplicaciones de matrices y convoluciones, donde existen muchas operaciones independientes o parcialmente independientes que pueden ejecutarse en paralelo.

Además del paralelismo, la reutilización de datos es un aspecto clave. En una multiplicación matricial, un mismo dato de entrada puede participar en varios productos antes de dejar de ser útil. Los arreglos sistólicos aprovechan

esta propiedad haciendo que los datos se desplacen entre PEs vecinos, evitando leerlos repetidamente desde memoria externa. Esta organización reduce la cantidad de accesos a memoria, disminuye la demanda de ancho de banda y mejora la eficiencia energética del acelerador [9].

Por esta razón, la optimización de un arreglo MAC no consiste solamente en colocar más unidades de cómputo, sino en mantenerlas ocupadas la mayor parte del tiempo. Si los PEs no reciben datos a tiempo, el arreglo puede quedar subutilizado aunque tenga una gran cantidad de recursos disponibles. Por eso, el diseño debe balancear la cantidad de MACs, el movimiento de datos y la capacidad de alimentar el arreglo sin generar esperas innecesarias.

En implementaciones recientes también se optimiza el paralelismo dentro de las propias unidades de cómputo. Por ejemplo, HMSA propone mejorar el uso de los recursos DSP en FPGA mediante técnicas de empaquetamiento de datos, permitiendo que un mismo recurso pueda ejecutar más multiplicaciones en paralelo según la combinación de precisión utilizada. Esto muestra que la optimización puede ocurrir tanto a nivel del arreglo completo como dentro de cada PE o unidad MAC [10].

V-C. Flujo de datos en arreglos MAC

El flujo de datos define la forma en que los operandos y los resultados parciales se mueven dentro del arreglo MAC. Esta decisión es importante porque determina qué datos permanecen más tiempo dentro de los PEs, cuáles se desplazan entre unidades vecinas y cuántas veces se debe acceder a la memoria. En arreglos sistólicos, el flujo de datos suele organizarse de manera regular, aprovechando la comunicación local entre PEs para reducir movimientos innecesarios.

En la literatura se suelen identificar tres estrategias principales de flujo de datos: *Output Stationary*, *Weight Stationary* e *Input Stationary*. En *Output Stationary*, cada PE mantiene un resultado parcial hasta completar la acumulación correspondiente. En *Weight Stationary*, los pesos permanecen dentro de los PEs durante más tiempo, mientras los datos de entrada se desplazan por el arreglo. En *Input Stationary*, ocurre lo contrario: los datos de entrada se mantienen localmente y los pesos o resultados parciales se mueven según la organización del cálculo [9].

La elección del flujo de datos afecta directamente el rendimiento y la eficiencia del acelerador. Por ejemplo, si una estrategia permite reutilizar más veces un dato dentro del arreglo, se reducen los accesos a memoria y se mejora el aprovechamiento de los PEs. Sin embargo, no existe un único flujo óptimo para todos los casos, ya que la mejor opción depende del tipo de operación, del tamaño de las matrices, de la cantidad de memoria disponible y de las restricciones del hardware.

Un ejemplo práctico de esta decisión aparece en HMSA, donde se adopta una arquitectura *Output Stationary* para simplificar el control y mantener los resultados parciales dentro del arreglo durante la operación. Además, el diseño organiza el ingreso de datos mediante mecanismos de retardo

y particionamiento, de forma que los operandos lleguen a los PEs en el momento correcto. Esto muestra que el flujo de datos no solo es una idea teórica, sino una decisión concreta de diseño que influye en el rendimiento final del acelerador [10].

V-D. Precisión numérica y eficiencia

La precisión numérica es otro factor importante en la optimización de arreglos MAC. En muchas aplicaciones de inteligencia artificial no siempre es necesario utilizar formatos de alta precisión, como FP32, para todas las operaciones. En su lugar, se pueden usar representaciones de menor cantidad de bits, como INT8, INT4 o incluso precisiones menores, con el objetivo de reducir el costo computacional, el consumo de memoria y la cantidad de datos que deben moverse dentro del acelerador.

Esta reducción de precisión puede mejorar la eficiencia porque una operación con menos bits requiere menos recursos de hardware y menor ancho de banda. Además, en ciertos casos permite ejecutar más multiplicaciones en paralelo dentro de una misma unidad de cómputo. Sin embargo, disminuir la precisión también puede afectar la exactitud del modelo, por lo que debe existir un equilibrio entre rendimiento y pérdida de precisión. Por esta razón, la cuantización no debe verse solo como una reducción arbitraria de bits, sino como una decisión de diseño que depende tanto del modelo como de la arquitectura utilizada [9].

Un ejemplo reciente de esta idea es HMSA, donde se propone un acelerador sistólico heterogéneo de precisión mixta para FPGA. En este diseño se optimiza el uso de los recursos DSP mediante técnicas de empaquetamiento de datos, de forma que una misma unidad pueda realizar más operaciones de multiplicación dependiendo de la combinación de bits usada en pesos y activaciones. Además, se combinan unidades basadas en DSP con unidades basadas en LUT, llamadas D-MAC y L-MAC, para aprovechar mejor los recursos disponibles en la FPGA [10].

Este tipo de optimización muestra que la eficiencia no depende únicamente de reducir la precisión, sino de adaptar esa precisión al hardware real. Si una combinación de bits reduce la exactitud del modelo pero no mejora la velocidad de inferencia, entonces no representa una buena optimización. Por eso, HMSA utiliza un modelo costo-rendimiento para seleccionar configuraciones de precisión que aumenten el rendimiento práctico del acelerador sin degradar innecesariamente la precisión del modelo [10].

V-E. Balance entre cómputo, memoria y comunicación

En un arreglo MAC, el rendimiento no depende solamente de la cantidad de unidades de cómputo disponibles. También depende de la capacidad del sistema para alimentar esas unidades con datos de forma constante. Si el arreglo puede realizar muchas operaciones por ciclo, pero los datos no llegan a tiempo desde memoria, los PEs deben esperar y el rendimiento real queda por debajo del rendimiento teórico.

Por esta razón, el diseño de arreglos MAC debe balancear tres aspectos: cómputo, memoria y comunicación. El cómputo está relacionado con la cantidad de operaciones MAC que pueden ejecutarse en paralelo. La memoria incluye tanto los buffers internos como los accesos a memoria externa. La comunicación se refiere al movimiento de datos entre memoria, buffers y PEs. Un diseño eficiente busca que los datos se reutilicen dentro del arreglo antes de volver a acceder a memoria externa, reduciendo así latencia, consumo energético y presión sobre el ancho de banda [9].

Este balance también depende del tamaño del arreglo y de los buffers disponibles. Un arreglo más grande puede ofrecer mayor paralelismo, pero también requiere más datos para mantenerse ocupado. De igual forma, buffers más grandes pueden reducir accesos a memoria externa, pero aumentan el consumo de área y energía. Por eso, el diseño óptimo no siempre es el de mayor tamaño, sino aquel que logra una relación adecuada entre recursos disponibles, flujo de datos y carga de trabajo.

En HMSA se observa este tipo de optimización a nivel práctico. El acelerador utiliza una arquitectura sistólica heterogénea basada en recursos DSP y LUT, junto con buffers de entrada, pesos y resultados para reducir accesos frecuentes a memoria externa. Además, emplea mecanismos como particionamiento en *tiles*, planificación tipo *ping-pong* y prelectura de datos para mantener el flujo de información hacia el arreglo. Esto permite mejorar el uso de los recursos de cómputo y reducir esperas durante la inferencia [10].

V-F. Ejemplo conceptual de optimización

Como ejemplo conceptual, puede considerarse un arreglo MAC utilizado para acelerar una multiplicación de matrices en una red neuronal. En una implementación básica, cada PE ejecuta una operación MAC y acumula resultados parciales. Sin embargo, si todos los PEs reciben datos directamente desde memoria externa, el sistema puede quedar limitado por el ancho de banda disponible. En ese caso, aunque existan muchas unidades MAC, no todas estarán ocupadas de manera constante.

Una primera optimización consiste en organizar el cálculo mediante un flujo de datos que permita reutilizar operandos dentro del arreglo. Por ejemplo, si un dato de entrada o un peso puede ser usado por varios PEs vecinos antes de salir del arreglo, se reducen los accesos a memoria y se mejora la eficiencia energética. Esta idea es la base de los arreglos sistólicos: los datos no solo se leen y se procesan una vez, sino que se desplazan ordenadamente entre unidades de procesamiento para aprovechar mejor cada acceso a memoria [9].

Una segunda optimización consiste en ajustar la precisión numérica al tipo de operación y al hardware disponible. En lugar de utilizar la misma cantidad de bits para todas las operaciones, se pueden usar precisiones menores o mixtas en pesos y activaciones. Esto permite reducir el tamaño de los datos, disminuir el tráfico de memoria y, en algunos casos, ejecutar más multiplicaciones en paralelo dentro de

una misma unidad física. En HMSA, por ejemplo, se propone una arquitectura heterogénea que combina unidades D-MAC basadas en DSP y unidades L-MAC basadas en LUT, junto con un modelo costo-rendimiento para seleccionar configuraciones de mayor eficiencia [10].

De esta forma, la optimización de arreglos MAC puede entenderse como una combinación de decisiones arquitectónicas: distribuir operaciones en paralelo, reutilizar datos, escoger un flujo de datos adecuado, balancear memoria y cómputo, y adaptar la precisión numérica al hardware. El objetivo final no es solamente aumentar la cantidad de MACs disponibles, sino lograr que el arreglo trabaje de forma sostenida, con menos esperas, menor movimiento de datos y mejor aprovechamiento de los recursos del acelerador.

REFERENCIAS

- [1] Y. Liao and C. Papachristou, "Systolic array for neural network acceleration," in *Proc. 2025 IEEE National Aerospace and Electronics Conference (NAECON)*, 2025.
- [2] A. Pușcașu, C. B. Ciobanu, and O. Buiu, "Systolic array matrix multiplication accelerator," in *Proc. 2024 International Semiconductor Conference (CAS)*, Sinaia, Romania, 2024, pp. 207–210.
- [3] Z. Feng, H. Zhang, A. Li, L. Xing, D. Wu, and Y. Liu, "A tensor core architecture for large-scale matrix multiplication operations on gpgpu processor," in *2025 4th International Symposium on Semiconductor and Electronic Technology (ISSET)*. IEEE, 2025, pp. 246–249.
- [4] A. Nada, G. M. Sarda, and E. Lenormand, "Cooperative warp execution in tensor core for risc-v gpgpu," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1424–1436.
- [5] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA '23)*. ACM, 2023, pp. 1–14.
- [6] D. N. Devi, G. A. Kumar, B. Gowda, and M. Rao, "Integrated MAC-based systolic arrays: Design and performance evaluation," in *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI '24)*. ACM, 2024, pp. 1–4.
- [7] A. J. Abdelmaksoud, S. Agwa, and T. Prodromakis, "DiP: A scalable, energy-efficient systolic array for matrix multiplication acceleration," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 73, no. 1, pp. 283–296, 2026.
- [8] P. Palacios, R. Medina, J.-L. Rouas, G. Ansaloni, and D. Atienza, "Systolic arrays and structured pruning co-design for efficient transformers in edge systems," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*. ACM, 2024.
- [9] R. Xu, S. Ma, Y. Guo, and D. Li, "A survey of design and optimization for systolic array-based dnn accelerators," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–37, Aug. 2023.
- [10] Y. Cao, H. Jiang, H. Li, Y. Tang, D. Shi, and G. Zhao, "Hmsa: High-performance heterogeneous mixed-precision cnn systolic array accelerator on fpga," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5s, pp. 1–27, Sep. 2025.